Vertex cover -> Un graf G cu N noduri si un numar K, se poate gasi o submultime S de k noduri a.i. orice muchie are cel putin un capat in submultimea S?

Independent set -> Un graf G cu N noduri si un numar K, se poate gasi o submultime S de k noduri a.i. orice muchie are cel mult un capat in submultimea S?

V-S = independent set

Vertex cover: G, k -> Independent set -> acelasi graf G, n-k

Daca pot construi un vertex cover de dimensiune k, se poate construi si un independent set de dimensiune n-k pe acelasi graf (demonstratia se face prin reducere la absurd)

3_colorare <= 4_colorare

K-coloare : G, k, se poate colora graful cu k culori a.i. sa nu avem 2 noduri adiacente de aceeasi culoare?

3_colorare :k =3

4_colorare: k =4

```
K-colorare(G,k)
{
    culori[N] = {0}
    for i=1:N
        culori[i] = choice(1..k)
    for (u,v) in G.Edges:
        if culori[u] == culori[v]:
            fail()
    success()
}
```

3_colorare(i) =1 <=> 4_colorare(F(i)) = 1

Functia de transformare: in graful initial se mai aduga nodul N+1, un nod special care e conectat la absolut toate celelale noduri. Acesta va fi inputul pentru 4-colorare.

3_colorare(i) = 1 => in graful G' nodurile se vor la fel, cu exceptia nodului N+1 care trebuie sa aiba cea de-a 4-a culoare

4_colorare(i) = 1 => in graful G' nodurile se vor colora a.i. nodul N+1 va avea o culoare diferita de restul => nodurile de la 1 la N vor avea nevoie doar de 3 culori

k_card(G) – cel mai mic numar k a.i. exista k submultimi de noduri V1,V2...Vk cu proprietatile:

1. Reuniune de Vi = V

2. Interesctie intre Vi, Vj = multimea vida

3. Pentru o submultime Vi, nu avem muchii intre nodurile din Vi

Dandu-se un graf G (M,N) si un numar k, este k = k_card(G) ?

```
K_Card_aux(G,N,k)

sizes[N]=  {0}

V[k][N] = {0}

marcat[N] = {0}

for i=1:N:

        sizes[i] = choice(1..N)

if sum(sizes)!=N:

        fail()

for i=1:k

        for j=1:sizes[i]:

                V[i][j] = choice(1..N)

                if marcat[V[i][j]] == true:
```

```
                    fail()
                marcat[V[i][j]] = true

for i=1:k
    for j=1:sizes[i]:
        for k=1:sizes[i]:
            if (V[i][j],V[i][k]) in G.Edges:
                fail()
success()

K_Card_Aux2(G,N,k)
subs[N] = {0}
for i=1:N
    subs[i] = choice(1..k)
V = []
for i=1:k
    V.append([])
for i=1:N
    V[subs[i]].append(i) // la subs[i] il adaug pe i
for i=1:k
    for j=1:V[i].size():
        for k=1:V[i].size():
            if (V[i][j],V[i][k]) in G.Edges:
                fail()
success()
// O(k*N^2)
```

```
K_Card_Aux3(G,N,k)

subs[N] = {0}

for i=1:N
        subs[i] = choice(1..k)

V = []

for (u,v) in G.Edges:

                if subs[u] == subs[v]:

                        fail()

success()
```

O(N + M)

```
K_Card(G,N,k)

        for i=1:k-1

                if K_Card_Aux3(G,n,i) == 1:

                        fail()

        return K_Card_Aux3(G,N,k)
```

SAT -> se da o expresie E care contine M termeni. Cei M termeni sunt conectati prin "and", iar in cadrul unui termen avem variabile conectate prin "or" (forma normala conjunctiva)

E= (x1 or x2 or x3 or x4) and (~x1 or x2 or x3 or x4) and (~x1 or ~x2 or x3) and (~x2 or x3 or ~x4)

Se pot gasi valori true/false pentru cele N variabile a.i. E(x1,x2,..xn) = true?

E = [[1 2 3 4] [-1 2 3 4] [-1 -2 3] [-2 3 -4]]

```
SAT (E, N)
vals[N] = {0}
for i=1:N
        vals[i] = choice{0,1}
for i=1:E.size():
        ok = 0
        for i=1:E[i].size():
                if E[i][j] > 0 and vals[|E[i][j]|] == 1:
                        ok = 1
                if E[i][j] < 0 and vals[|E[i][j]|] == 0:
                        ok = 1
        if ok ==0 :
                fail()
success()
```